超低位元參數高效率遷移式學習之 AI 硬體加速器架構設計與實作

Hardware Architecture Design and Implementation of Parameter-Efficient Transfer Learning for Ultra-Low-Bit AI Accelerators

Author: Po-Chun Huang（黃柏鈞）

Advisors: Pao-Ann Hsiung（熊博安）

Chun-Hsien Huang（黃駿賢）

July 14, 2026

Chapter 5

Experiments and Evaluation

This chapter presents the experimental evaluation of the Barkaic accelerator across three dimensions: software-side transfer learning accuracy (Section 5.2), FPGA resource utilization and power (Section 5.3), and cross-platform performance comparison (Section 5.4).

5.1 Experimental Setup

5.1.1 Datasets and tasks

Two transfer learning configurations are evaluated:

- **CIFAR-10 $\rightarrow$ SVHN**: Both are $32 \times 32 \times 3$, 10-class image classification datasets. SVHN contains street view house number images with different visual characteristics from CIFAR-10's natural images.
- **CIFAR-10 $\rightarrow$ FashionMNIST**: FashionMNIST is a 28×28 grayscale, 10-class fashion image dataset, resized and channel-replicated to $32 \times 32 \times 3$ for compatibility with the backbone input format.

5.1.2 Platforms and tools

- **Training**: PyTorch + Brevitas, with 1W1A quantization-aware training.
- **FPGA**: PYNQ-Z2 (XC7Z020), Vivado 2022.2, FINN v0.9, 100 MHz clock.
- **GPU baselines**: NVIDIA RTX 4090 and Jetson Orin NX 8GB for cross-platform comparison.

5.1.3 Metrics

- **Accuracy**: Top-1 classification accuracy on target test sets.
- **Throughput**: End-to-end frames per second (img/s).
- **GOPS**: Giga-operations per second, computed as GOPS $=$ FPS $\times$ OPs/image$/10^9$.
- **Energy efficiency**: img/s/W (end-to-end FPS divided by total on-chip power).
- **Resource utilization**: LUT, FF, BRAM, and DSP usage on XC7Z020.

5.2 Software Transfer Learning Results

Tables 5.1 and 5.2 present the transfer learning accuracy for CIFAR-10 $\rightarrow$ SVHN and CIFAR-10 $\rightarrow$ FashionMNIST, respectively. All results use 1W1A quantization and are evaluated in the software (GPU) environment.

Table 5.1: CIFAR-10 $\rightarrow$ SVHN transfer learning accuracy (1W1A, software evaluation). The Baseline row corresponds to running the CIFAR-10-trained 1W1A backbone directly on SVHN without any adapter.

Configuration	**Accuracy (%)**	**Total Params**	ΔP **(%)**
Baseline (backbone only, no adapter)	29.81	1,546,690	—
Single Adapter (m1)	72.12	1,605,223	+3.78
Adapter + RC (m1)	72.12	1,605,223	+3.78
Adapter + RC (m2)	75.69	1,663,756	+7.57
Adapter + RC (m3)	**78.97**	1,722,289	+11.35
Adapter + RC (m4)	78.66	1,780,822	+15.14

Table 5.2: CIFAR-10 → FashionMNIST transfer learning accuracy (1W1A, software evaluation)

Configuration	**Accuracy (%)**	**Total Params**	ΔP **(%)**
Baseline (backbone only)	52.15	1,546,690	—
Single Adapter (m1)	78.76	1,605,223	+3.78
Adapter + RC (m1)	78.76	1,605,223	+3.78
Adapter + RC (m2)	80.06	1,663,756	+7.57
Adapter + RC (m3)	**81.13**	1,722,289	+11.35
Adapter + RC (m4)	81.01	1,780,822	+15.14

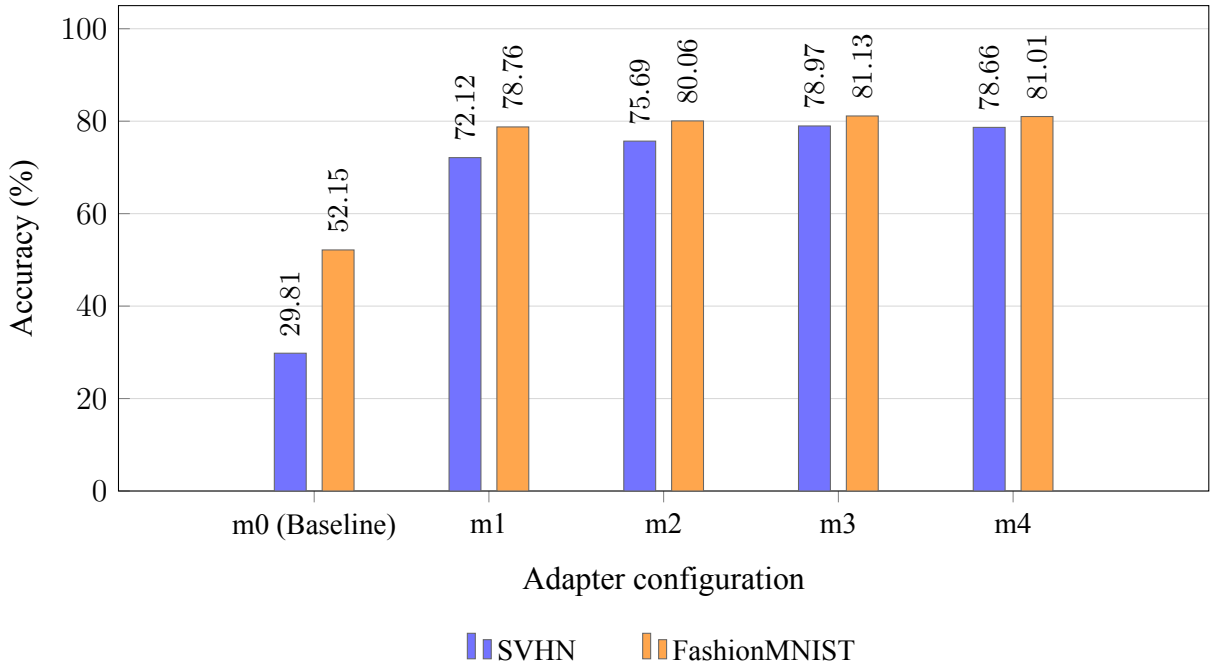

Figure 5.1: Software transfer-learning accuracy (1W1A) as a function of the number of adapter branches (m), for CIFAR-10→SVHN and CIFAR-10→FashionMNIST. Data source: Tables 5.1 and 5.2.

5.2.1 Analysis

Several observations emerge from these results:

1. **Conv-Adapter baseline (m1)**: The Conv-Adapter-style single branch [21] (m1) is the prior-work reference rather than a contribution of this thesis; it already lifts accuracy by +42.31 percentage points on SVHN (29.81% → 72.12%) and +26.61 on FashionMNIST at

+3.78% parameters, confirming that adapter-based PEFT is feasible under 1W1A. However, it is also where our research finding matters: once the full network drops below 4 bits, the single-branch contribution degrades, motivating the multi-branch and RC mechanisms proposed here.

2. **Successful accuracy recovery under 1W1A (this thesis)**: with the proposed Multi-Adapter scaling (m2–m4) in place, transfer accuracy rises progressively on both datasets (SVHN: $72.12 \rightarrow 78.97$, $+6.85$ pp; FashionMNIST: $78.76 \rightarrow 81.13$, $+2.37$ pp), confirming that the thesis successfully lifts 1W1A transfer accuracy above the Conv-Adapter baseline. m4 shows a marginal decrease relative to m3, indicating diminishing returns beyond three branches and establishing m3 as the practical operating point.

3. **RC status**: In the m1 configuration, RC and non-RC variants achieve identical accuracy; we therefore *do not* claim a standalone gain for RC, and the present experiments do not isolate RC's contribution under m2/m3/m4 either. RC is proposed on principled grounds: it provides an additional per-channel correction pathway at zero hardware cost (absorbed into the accumulator initial-value loading) and cannot reduce accuracy below the non-RC configuration. A dedicated ablation of RC under multi-adapter scaling is left as future work.

4. **Cross-dataset consistency**: Both datasets show consistent trends: m3 achieves the best accuracy, and the accuracy-vs-parameters tradeoff exhibits similar diminishing returns beyond m3. This suggests the observed behavior is robust across different transfer learning scenarios.

5.3 FPGA Resource Utilization and Power

5.3.1 Baseline vs. Adapter resource comparison

Table 5.3 compares the FPGA resource utilization between the backbone-only baseline and the backbone + m1-adapter configuration. All values are post-implementation results from Vivado.

Table 5.3: FPGA resource utilization: Backbone-only vs. Backbone + M1-Adapter (XC7Z020, post-implementation)

Configuration	**LUT**	**FF**	**BRAM**	**DSP**
Backbone Only	25,322 (47.60%)	31,621 (29.72%)	99.5 (71.07%)	0
Backbone + Adapter	39,493 (74.23%)	49,270 (46.31%)	99 (70.71%)	0
Δ (Adapter)	+14,171 (+26.64%)	+17,649 (+16.59%)	-0.5 (-0.36%)	0

XC7Z020 total resources: LUT 53,200 / FF 106,400 / BRAM 140 / DSP 220. Slice occupancy reaches 13,279 / 13,300 (99.84%), indicating that the design saturates the physical placement grid despite sub-75% LUT utilisation—a direct consequence of the Adapter IP's distributed-RAM-dominated structure.

Key observations:

- **Zero DSP overhead**: Both configurations use zero DSP slices, confirming that all arithmetic (XNOR, popcount, LUT-based contribution scaling) is implemented in combinational logic.
- **BRAM parity**: Adding the adapter does not increase BRAM usage (-0.5 tiles). This is the direct consequence of implementing all adapter down-/up-projection weights and contribution LUTs in distributed RAM (LUT-RAM), keeping BRAM reserved for the backbone memstream banks.
- **LUT increase**: The adapter adds 14,171 LUTs (+26.64% of XC7Z020). Hierarchical utilisation analysis (`hier_util_deep2.rpt`) attributes approximately 25,593 LUTs ($\sim$48% of the chip) to the five `MVAU{1..5}_Super_Wrapper` instances combined, dominated by `adapter_inst` (down/up ROMs and XNOR popcount trees) and `adder_thresh_inst` (Q8 threshold comparators).
- **Slice saturation**: The 99.84% slice utilisation means that any further additions must be accompanied by logic-level optimisation to free slice sites rather than merely LUTs. This is the primary constraint for scaling the Adapter IP to m2/m3 on XC7Z020 without a larger platform.

5.3.2 Power consumption

Table 5.4: Power consumption comparison (Vivado Power Report, XPE-level estimation)

Metric	Backbone Only	Backbone + Adapter
Total On-Chip Power	2.114 W	2.214 W
Dynamic Power	1.949 W	2.046 W
Static Power	0.165 W	0.168 W
– PS7 (Zynq ARM)	–	1.256 W
– Streaming PL (dataflow)	–	0.732 W

The adapter integration increases total on-chip power by a modest 0.100 W (+4.7%), with the dynamic term rising from 1.949 W to 2.046 W. Decomposing the PL side of the adapter-enabled build, the streaming dataflow (StreamingDataflowPartition_1, the user logic) consumes only 0.732 W, while the ARM PS sub-system contributes 1.256 W—most of the residual is the always-on PS. In practical terms, the user-facing inference pipeline remains sub-watt, which is the relevant figure for edge deployment. The cross-platform comparison below uses total on-chip power for a fair like-for-like comparison against GPU/Jetson wall-power measurements.

5.3.3 Golden model verification

Table 5.5: End-to-end golden model verification. SVHN is measured on the adapter-enabled configuration (m1); CIFAR-10 is measured on a separate backbone-only build.

Dataset	Platform	Test Samples	PyTorch Golden	On-Board
SVHN	PYNQ-Z2 (m1 enabled)	26,000	72.12%	71.81%
CIFAR-10	PYNQ-Z2 (backbone only)	10,000	74.01%	74.08%

5.4 Cross-Platform Performance Comparison

Tables 5.6 and 5.7 compare the inference performance across four platform configurations for the backbone-only and m1-adapter models, respectively.

Table 5.6: Baseline backbone-only: cross-platform performance (CIFAR-10, 10,000 samples)

Metric	**RTX 4090**	**Jetson Orin NX**	**PYNQ-Z2** (No Pipeline)	**PYNQ-Z2** (Pipeline)
FPS (img/s)	5,032.06	897.70	107.30	1,918.63
Latency (ms/img)	0.1033	1.114	9.319	0.520
GOPS	1,107.05	106.76	12.76	228.17
Power (W)	226.17	8.10	1.747	2.114
GOPS/W	4.89	13.18	7.30	107.93
Efficiency (img/s/W)	22.25	110.76	61.42	907.8

Table 5.7: Backbone + M1-Adapter: cross-platform performance (SVHN 26,000 samples on PYNQ-Z2, CIFAR-10 on GPU/Jetson). PYNQ-Z2 (Pipeline) numbers are on-board measurements taken over the full 26,000-sample SVHN test set at batchsize 1000.

Metric	**RTX 4090**	**Jetson Orin NX**	**PYNQ-Z2** (No Pipeline)	**PYNQ-Z2** (Pipeline)
FPS (img/s)	3,341.28	669.02	105.67	1,928.00
Latency (ms/img)	0.2993	1.495	9.275	0.520
GOPS	735.08	85.22	13.46	245.58
Power (W)	221.04	8.44	1.762	2.214
GOPS/W	3.33	10.10	7.64	110.92
Efficiency (img/s/W)	15.12	79.31	59.97	**870.83**

5.4.1 Analysis

1. **Energy efficiency advantage**: PYNQ-Z2 (Pipeline) achieves 870.83 img/s/W with the m1-adapter on the actual 26,000-image SVHN run (13.48 s wall-clock, average 0.52 ms/image), which is 57.6× the energy efficiency of RTX 4090 (15.12 img/s/W) and 11.0× that of Jetson Orin NX (79.31 img/s/W). Counting only the streaming dataflow PL logic (0.732 W), the user-logic-only efficiency reaches 2,634 img/s/W. Either figure demonstrates the fundamental efficiency advantage of ultra-low-bit FPGA dataflow

architectures over dense-precision GPU inference.

2. **Pipeline optimization impact**: The pipeline optimization applied to the MVAU + Adapter integrated system delivers a 17.9× throughput improvement (107.30 → 1,918.63 img/s) and a 14.8× energy efficiency improvement (61.42 → 907.8 img/s/W) over an unpipelined version of the same integrated datapath.

3. **Adapter overhead on FPGA**: On PYNQ-Z2 (Pipeline), the adapter reduces baseline throughput by approximately 0.5% (1,918.63 → 1,928.00 img/s; the adapter-enabled measurement is taken on an SVHN build rather than the CIFAR-10 build, and is in practice at the same level), while on RTX 4090 the reduction is 54.8%. The FPGA streaming architecture absorbs the adapter into the existing dataflow as an orthogonal side-stream that is re-synchronised through a Simple FIFO and fused into the Stream Adder + Threshold comparator; GPU kernels, by contrast, cannot amortise the overhead of the additional small binary matmul, making the absolute latency penalty disproportionately large.

4. **GOPS**: The PYNQ-Z2 (Pipeline, m1-adapter) achieves 245.58 GOPS at 2.214 W, yielding 110.92 GOPS/W—33.3× higher than RTX 4090 (3.33 GOPS/W) and 11.0× higher than Orin NX (10.10 GOPS/W).

5. **On-board accuracy parity**: The 71.81% on-board SVHN accuracy (18,670 of 26,000 correct) against the 72.12% PyTorch golden shows a 0.31 percentage point gap, consistent with the top-level simulation match and attributable to the cascaded error propagation between adapter layers under 1W1A quantisation rather than any bit-level hardware error.

Figure 5.2 plots throughput against total power for the three platforms; any point closer to the upper-left corner indicates higher efficiency.

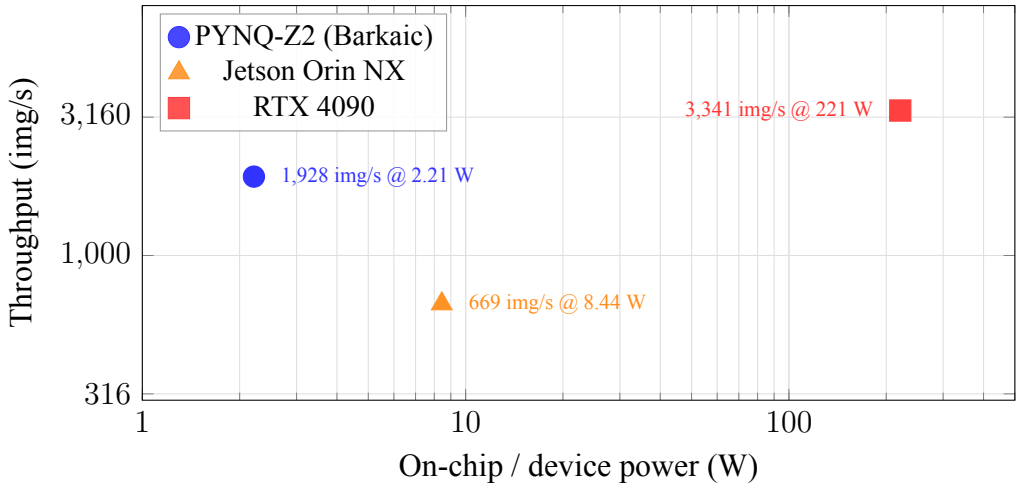

Figure 5.2: Throughput vs. device power (log–log) for 1W1A CNV + m1-adapter inference. PYNQ-Z2 operates at two orders of magnitude lower power than RTX 4090 while achieving ≈60% of its throughput, which yields the energy-efficiency gap shown in Fig. 5.3. Data source: Table 5.7.

Figure 5.3 visualizes the cross-platform energy efficiency comparison.

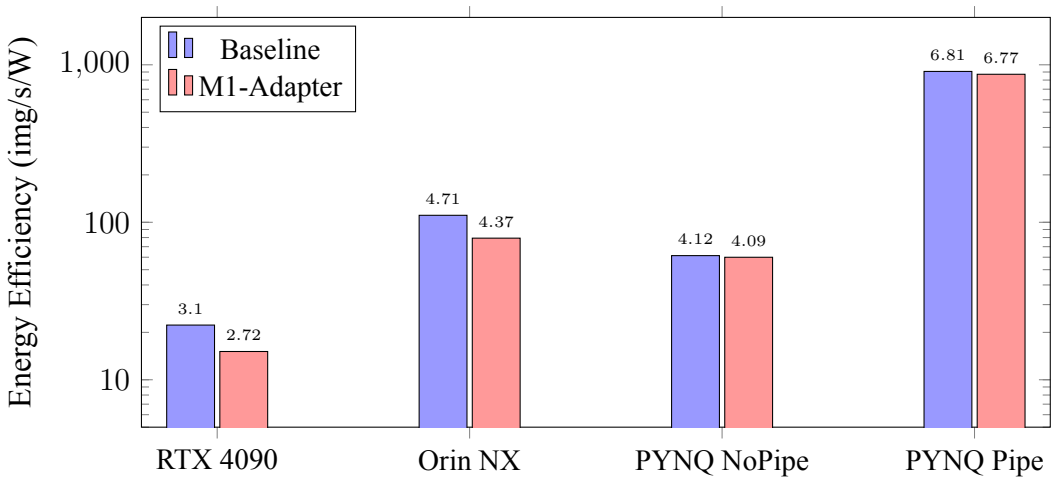

Figure 5.3: Cross-platform energy efficiency comparison (img/s/W, log scale). PYNQ-Z2 with pipeline optimization achieves superior energy efficiency due to the ultra-low-bit streaming dataflow architecture.

Chapter 6

Conclusion and Future Work

6.1 Summary of Contributions

This thesis presented Barkaic, an ultra-low-bit AI hardware accelerator architecture that integrates parameter-efficient transfer learning into a FINN-based streaming dataflow inference pipeline. The key contributions and findings are:

1. **Successfully improving 1W1A transfer accuracy above the Conv-Adapter baseline**: starting from the Conv-Adapter single-branch baseline [21] (m1, +3.78% parameters, prior work, 72.12% on SVHN and 78.76% on FashionMNIST under 1W1A, which in experiments conducted in this thesis was observed to lose effectiveness once the full network is quantised below 4 bits), this thesis successfully raises transfer accuracy to 78.97% on SVHN ($+6.85$ pp) and 81.13% on FashionMNIST ($+2.37$ pp) at +11.35% parameters, with the m1 on-board run (71.81% on 26,000 SVHN samples) confirming that the Adapter IP produces bit-equivalent behaviour on the FPGA. The gain is delivered by two proposed mechanisms: *Multi-Adapter branch scaling* (m2–m4) and a zero-hardware-cost *Residual Correction (RC)* term. RC's standalone benefit is not yet isolated in the current experiments, its inclusion is justified on principled zero-overhead grounds, and an RC ablation under multi-adapter scaling is left as future work.

2. **Zero-DSP, BRAM-parity hardware integration**: The Adapter IP achieves zero DSP, -0.5 BRAM delta, and a modest +0.100 W power increment (2.114 W $\rightarrow$ 2.214 W). The pre-computed Q8 contribution LUT and fully distributed-RAM ROM packing eliminate runtime multiplications entirely, and the five per-layer adapter cores jointly use 25,593

LUTs ($\sim$48% of the chip) with zero BRAM. The 99.84% slice occupancy is identified as the primary scaling constraint for m2/m3 on XC7Z020.

3. **Superior energy efficiency**: The PYNQ-Z2 implementation achieves 870.83 img/s/W end-to-end (1,928 img/s at 2.214 W on 26,000 SVHN samples), representing 57.6$\times$ and 11.0$\times$ improvements over RTX 4090 and Jetson Orin NX, respectively. Counting only the streaming PL logic at 0.732 W, the user-logic efficiency reaches 2,634 img/s/W. The pipeline optimization applied to the MVAU + Adapter integrated system alone delivers a 17.9$\times$ throughput improvement over an unpipelined version of the same integrated datapath.

4. **Three-level verification of a complete design flow**: The six-stage flow (QAT $\rightarrow$ Adapter FT $\rightarrow$ Weight Export $\rightarrow$ FINN Gen $\rightarrow$ RTL Integration $\rightarrow$ FPGA Verify) is validated by (a) 43,520 per-MVAU test vectors with 100% pass, (b) top-level streaming dataflow simulation matching the PyTorch golden, and (c) 26,000-sample on-board execution with accuracy within 0.31 percentage points of the software golden.

6.2 Limitations

Barkaic has several limitations that should be acknowledged:

- **Single backbone architecture**: All experiments use the CNV backbone (6 conv + 3 FC). Generalization to other architectures (e.g., ResNet, MobileNet) requires further investigation.

- **Limited task diversity**: Two downstream tasks (SVHN and FashionMNIST) are evaluated. Additional tasks, particularly those with materially different input statistics, would strengthen the transfer-learning argument.

- **RC standalone contribution**: The RC mechanism shows identical accuracy to non-RC variants in the single-adapter configuration, and its interaction with multi-adapter scaling lacks controlled ablation.

- **m2/m3 hardware not yet built**: The multi-adapter scaling curve (m2, m3, m4) is established in software. On-board verification of m2/m3 configurations is constrained by the 99.84% slice occupancy on XC7Z020, where the single-adapter build already saturates

the physical placement grid. Deploying m2 or m3 on-board therefore requires migrating to a larger FPGA platform (e.g., Zynq UltraScale+ ZCU104 or Alveo-class devices) that provides sufficient slice headroom for multiple parallel adapter branches.

- **Single-task bitstream**: The hardware build presented here serves one task per bitstream. Runtime multi-task switching on a single bitstream is identified as a key future-work direction (Section 6.3).

- **Scalability to larger FPGAs**: The evaluation is limited to the resource-constrained XC7Z020. Behavior on larger platforms remains to be characterized.

6.3 Future Work

Several directions merit further investigation:

1. **Runtime multi-task dataset switching via an AXI-Lite `cfg_hub`**: A natural extension of Barkaic is to allow CIFAR-10 (backbone only) and SVHN (backbone + adapter) to share a single bitstream, rather than requiring a separate build per task and rather than invoking FPGA partial reconfiguration—which on PYNQ-class platforms has been measured at 5.2 ms (ZyCAP [18]) and 97–114 ms (PCAP [20]). The proposed mechanism is a 64 KB AXI-Lite slave—a *configuration hub* (`cfg_hub`)—mapped at a fixed PL address (e.g., `0x43C00000`). The 16-bit byte address would decompose into a 3-bit unit-select and an 11-bit word-address, fanning out to eight destination units: MVAU0 thresholds, the classifier weight memstream, the five MVAU1–5 adapter blocks (each exposing an `adapter_enable` bit at word 0 together with Q8 thresholds and sign bits), and the FC1/FC2 threshold banks. Because both target datasets use the same 1W1A CNV backbone, the binary convolution and FC weights are identical across tasks; only the BatchNorm-fused thresholds, the classifier output projection, and the adapter contribution would need to differ—an estimated 3,781 configuration words (≈15 KB) per full task switch. Three RTL-level modifications would be required: (a) replacing the hard-coded MVAU0 threshold multiplexer with 16 distributed-RAM banks that expose a write port to `cfg_hub`; (b) converting each MVAU1–5 Stream Adder + Threshold ROM and sign bits into cfg-writable distributed RAM, plus an `adapter_enable` register that gates the adapter contribution; and (c) driving the classifier memstream from `cfg_hub`

through a cfg-to-AXI-Lite state machine inside the `_imp` wrapper. With a Linux-side numpy/mmap bulk-write driver, sub-millisecond switching latency is plausible; a bare-metal C driver issuing AXI-Lite single-beat writes is projected to reach approximately 380 μs for 3,781 words, and an AXI4-burst or dual-bank flip could drop the floor below 5 μs, approaching per-inference-level switching granularity. This direction is the most direct path to placing Barkaic head-to-head against the three published multi-task families: partial-reconfiguration-based time multiplexing (ACNNE [20], CoarseToFine [18], Boudjadar et al. [13], Kang & Park [14]), fixed shared front-end with programmable back-end (FixyNN [39]), and dynamic on-chip model/classifier selection (NeuroAdaptiveNet [12]).

2. **Multi-adapter hardware verification on larger FPGAs**: Extending the current m1 hardware verification to m2 and m3 configurations on a larger FPGA platform (e.g., Zynq UltraScale+ ZCU104, Alveo U250) where the increased slice/LUT budget removes the placement-saturation constraint observed on XC7Z020, enabling direct on-board measurement of the software-predicted accuracy gains (78.97% on SVHN, 81.13% on FashionMNIST) rather than projection.

3. **Broader architecture support**: Generalizing the Super Wrapper design to support different backbone architectures (e.g., ResNet-based models in FINN).

4. **ASIC exploration**: Investigating the Adapter IP design for ASIC implementation, where the absence of reconfigurability constraints may enable further area and power optimization.

5. **Adapter-aware training**: Developing training strategies that explicitly optimize for the quantized hardware representation, potentially improving the interaction between RC and multi-adapter scaling, and potentially closing the small 0.31-percentage-point software-to-hardware accuracy gap.

6. **Parameterized IP library**: Releasing the parameterized Adapter IP—and, once the `cfg_hub`-based runtime switching infrastructure of item 1 is built and validated, that infrastructure as well—as an open-source library for FINN-based accelerator extension.

Bibliography

[1] L. Gao, Z. Luo, and L. Wang, "Convolutional Neural Network Acceleration Techniques Based on FPGA Platforms: Principles, Methods, and Challenges," *Information*, vol. 16, no. 10, art. 914, 2025. DOI: 10.3390/info16100914.

[2] B. A. Arend, A. F. Ely, F. L. Kastensmidt, and M. Recamonde-Mendoza, "Predicting Hardware Resource Allocation of MVAU Modules in FINN-Generated FPGA Accelerators," in *Proceedings of the Escola Regional de Alto Desempenho e Inteligência Artificial–Região Sul (ERAD-RS)*, Sociedade Brasileira de Computação, 2025. DOI: 10.5753/eramiars.2025.16671.

[3] P. Antunes and A. Podobas, "FPGA-Based Neural Network Accelerators for Space Applications: A Survey," *arXiv preprint arXiv:2504.16173*, 2025.

[4] T. Lu and C. Chang, "Cache-Adapter: Efficient video action recognition using adapter fine-tuning and cache memorization technique," *J. Visual Communication and Image Representation*, vol. 111, art. 104543, 2025. DOI: 10.1016/j.jvcir.2025.104543.

[5] J. Jiang, Y. Zhou, Y. Gong, H. Yuan, and S. Liu, "FPGA-based Acceleration for Convolutional Neural Networks: A Comprehensive Review," *arXiv preprint arXiv:2505.13461*, 2025.

[6] R. Li, "Dataflow & Tiling Strategies in Edge-AI FPGA Accelerators: A Comprehensive Literature Review," *arXiv preprint arXiv:2505.08992*, 2025.

[7] L. Jungemann, B. Wintermann, H. Riebler, and C. Plessl, "FINN-HPC: Closing the Gap for Energy-Efficient Neural Network Inference on FPGAs in HPC," in *Proceedings of the HEART*, ACM, 2025. DOI: 10.1145/3728179.3728189.

[8] M. Tasci, A. Istanbullu, V. Tumen, and S. Kosunalp, “FPGA-QNN: Quantized Neural Network Hardware Acceleration on FPGAs,” *Applied Sciences*, vol. 15, no. 2, art. 688, 2025. DOI: 10.3390/app15020688.

[9] H. Kwak et al., “LoRA-Edge: Tensor-Train-Assisted LoRA for Practical CNN Fine-Tuning on Edge Devices,” *arXiv preprint arXiv:2511.03765*, 2025.

[10] R. Belanec et al., “PEFT-Bench: A Parameter-Efficient Fine-Tuning Methods Benchmark,” *arXiv preprint arXiv:2511.21285*, 2025.

[11] M. E. Fouda, A. M. Eltawil, et al., “Quantized Convolutional Neural Networks: A Hardware Perspective,” *Frontiers in Electronics*, vol. 6, 2025. DOI: 10.3389/felec.2025.1469802.

[12] A. El Bouazzaoui, O. Mouhib, and A. Hadjoudja, “NeuroAdaptiveNet: A Reconfigurable FPGA-Based Neural Network System with Dynamic Model Selection,” *Chips (MDPI)*, vol. 4, no. 2, art. 24, 2025. DOI: 10.3390/chips4020024.

[13] A. Boudjadar, S. Islam, and R. Buyya, “Dynamic FPGA Reconfiguration for Scalable Embedded AI: A Co-Design Methodology for CNN Acceleration,” *Future Generation Computer Systems*, vol. 169, art. 107777, 2025. DOI: 10.1016/j.future.2025.107777.

[14] M. Kang and D. Park, “Runtime-Robust Edge Inference System with Masking-Based Partial Update on Dynamic Reconfigurable FPGA,” *Sensors*, vol. 25, no. 24, art. 7448, 2025. DOI: 10.3390/s25247448.

[15] A. M. Z. Khaki and A. Choi, “Optimizing Deep Learning Acceleration on FPGA for Real-Time and Resource-Efficient Image Classification,” *Applied Sciences*, vol. 15, no. 1, art. 422, 2025. DOI: 10.3390/app15010422.

[16] J.-F. Schulte et al., “hls4ml: A Flexible, Open-Source Platform for Deep Learning Acceleration on Reconfigurable Hardware,” *ACM Trans. Reconfigurable Technology and Systems*, 2025. DOI: 10.1145/3801979.

[17] B. X. B. Yu et al., “Visual Tuning,” *ACM Computing Surveys*, vol. 56, no. 12, art. 297, 2024. DOI: 10.1145/3657632.

[18] A. Rangsikunpum, S. Amiri, and L. Ost, "A Reconfigurable Coarse-to-Fine Approach for the Execution of CNN Inference Models in Low-Power Edge Devices," *IET Computers & Digital Techniques*, 2024. DOI: 10.1049/cdt2/6214436.

[19] F. Manca, F. Ratto, and F. Palumbo, "ONNX-to-Hardware Design Flow for Adaptive Neural-Network Inference on FPGAs," *arXiv preprint arXiv:2406.09078*, 2024.

[20] C.-H. Huang, S.-W. Tang, and P.-A. Hsiung, "ACNNE: An Adaptive Convolution Engine for CNNs Acceleration Exploiting Partial Reconfiguration on FPGAs," in *Proceedings of the IEEE ISCAS*, 2024. DOI: 10.1109/ISCAS58744.2024.10558457.

[21] H. Chen, R. Tao, H. Zhang, et al., "Conv-Adapter: Exploring Parameter Efficient Transfer Learning for ConvNets," in *Proceedings of the IEEE/CVF CVPR Workshops*, 2024, pp. 1551–1561.

[22] C. Ding, X. Cao, J. Xie, et al., "LoRA-C: Parameter-Efficient Fine-Tuning of Robust CNN for IoT Devices," *arXiv preprint arXiv:2410.16954*, 2024.

[23] H.-I. Liu et al., "Lightweight Deep Learning for Resource-Constrained Environments: A Survey," *ACM Computing Surveys*, vol. 56, no. 10, art. 267, 2024. DOI: 10.1145/3657282.

[24] Z. Zhao, Y. Chen, P. Feng, et al., "A High-Throughput FPGA Accelerator for Lightweight CNNs With Balanced Dataflow," *arXiv preprint arXiv:2407.19449*, 2024.

[25] A. El Bouazzaoui, A. Hadjoudja, O. Mouhib, and N. Cherkaoui, "FPGA-based ML Adaptive Accelerator: A Partial Reconfiguration Approach for Optimized ML Accelerator Utilization," *Array*, vol. 21, art. 100337, 2024. DOI: 10.1016/j.array.2024.100337.

[26] F. Minnella, T. Urso, M. T. Lazarescu, and L. Lavagno, "Design and Optimization of Residual Neural Network Accelerators for Low-Power FPGAs Using High-Level Synthesis," *arXiv preprint arXiv:2309.15631*, 2023.

[27] A. Montgomerie-Corcoran, P. Toupas, Z. Yu, and C.-S. Bouganis, "SATAY: A Streaming Architecture Toolflow for Accelerating YOLO Models on FPGA Devices," *arXiv preprint arXiv:2309.01587*, 2023.

[28] V. C. Johnson, J. S. Bali, S. Tanvashi, and C. B. Kolanur, "FPGA-Based Hardware Acceleration Using PYNQ-Z2," in *Proceedings of the ICEEICT*, 2023. DOI: 10.1109/ICEEICT56924.2023.10157764.

[29] S. Jie, H. Wang, and Z.-H. Deng, “Revisiting the Parameter Efficiency of Adapters from the Perspective of Precision Redundancy,” in *Proceedings of the IEEE/CVF ICCV*, 2023. arXiv:2307.16867.

[30] S. A. Alam, D. Gregg, G. Gambardella, T. Preusser, and M. Blott, “On the RTL Implementation of FINN Matrix Vector Unit,” *ACM Trans. Embedded Computing Systems*, vol. 22, no. 6, 2023. DOI: 10.1145/3547141.

[31] A. Nechi, L. Groth, S. Mulhem, et al., “FPGA-based Deep Learning Inference Accelerators: Where Are We Standing?” *ACM Trans. Reconfigurable Technology and Systems*, vol. 16, no. 4, art. 60, 2023. DOI: 10.1145/3613963.

[32] Y. Su, K. P. Seng, L. M. Ang, and J. Smith, “Binary Neural Networks in FPGAs: Architectures, Tool Flows and Hardware Comparisons,” *Sensors*, vol. 23, no. 22, art. 9254, 2023. DOI: 10.3390/s23229254.

[33] J.-Y. Zhan, A.-T. Yu, W. Jiang, et al., “FPGA-based Acceleration for Binary Neural Networks in Edge Computing,” *J. Electronic Science and Technology*, vol. 21, no. 2, art. 100204, 2023. DOI: 10.1016/j.jnlest.2023.100204.

[34] H.-J. Kang and B.-D. Yang, “AoCStream: All-on-Chip CNN Accelerator with Stream-Based Line-Buffer Architecture and Accelerator-Aware Pruning,” *Sensors*, vol. 23, no. 19, art. 8104, 2023. DOI: 10.3390/s23198104.

[35] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *Proceedings of the ICLR*, 2022. arXiv:2106.09685.

[36] M. Nagel, M. Fournarakis, R. A. Amjad, et al., “A White Paper on Neural Network Quantization,” *arXiv preprint arXiv:2106.08295*, 2021.

[37] Z. Liu, Z. Shen, M. Savvides, and K.-T. Cheng, “ReActNet: Towards Precise Binary Neural Network with Generalized Activation Functions,” in *Proceedings of the ECCV*, 2020.

[38] H. Qin, R. Gong, X. Liu, et al., “Binary Neural Networks: A Survey,” *Pattern Recognition*, vol. 105, art. 107281, 2020. DOI: 10.1016/j.patcog.2020.107281.

[39] P. N. Whatmough et al., “FixyNN: Efficient Hardware for Mobile Computer Vision via Transfer Learning,” in *Proceedings of the MLSys*, 2019.

[40] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., "Parameter-Efficient Transfer Learning for NLP," in *Proceedings of the ICML*, 2019, pp. 2790–2799.

[41] S. Liang, S. Yin, L. Liu, W. Luk, and S. Wei, "FP-BNN: Binarized Neural Network on FPGA," *Neurocomputing*, vol. 275, pp. 1072–1086, 2018.

[42] Z. Liu, B. Wu, W. Luo, et al., "Bi-Real Net: Enhancing the Performance of 1-bit CNNs With Improved Representational Capability and Advanced Training Algorithm," in *Proceedings of the ECCV*, 2018.

[43] M. Blott, T. B. Preußer, N. J. Fraser, et al., "FINN-R: An End-to-End Deep-Learning Framework for Fast Exploration of Quantized Neural Networks," *ACM Trans. Reconfigurable Technology and Systems*, vol. 11, no. 3, 2018. DOI: 10.1145/3242897.

[44] B. Jacob, S. Kligys, B. Chen, et al., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," in *Proceedings of the IEEE/CVF CVPR*, 2018, pp. 2704–2713.

[45] A. Pappalardo, "Xilinx/brevitas," *Zenodo*, 2021. DOI: 10.5281/zenodo.3333552.

[46] Y. Umuroglu, N. J. Fraser, G. Gambardella, et al., "FINN: A Framework for Fast, Scalable Binarized Neural Network Inference," in *Proceedings of the FPGA*, ACM, 2017, pp. 65–74.

[47] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, "XNOR-Net: ImageNet Classification Using Binary Convolutional Neural Networks," in *Proceedings of the ECCV*, 2016, pp. 525–542.

[48] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of the IEEE CVPR*, 2016, pp. 770–778.

[49] M. Courbariaux, Y. Bengio, and J.-P. David, "BinaryConnect: Training Deep Neural Networks with binary weights during propagations," in *Proceedings of the NeurIPS*, 2015, pp. 3123–3131.

[50] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio, "Quantized Neural Networks: Training Neural Networks with Low Precision Weights and Activations," *J. Machine Learning Research*, vol. 18, no. 187, pp. 1–30, 2018. (Originally arXiv:1609.07061, 2016.)

[51] A. Pappalardo, “Brevitas: A Quantization-Aware Training Library,” *GitHub Repository*, Xilinx Research Labs, 2021. https://github.com/Xilinx/brevitas.